

MEMoE: Memory Expansion for Mixture-of-Experts Inference

When a slow memory tier can hold the experts, and when it cannot

Team King Bob

Shashank Yammanuru Shreyansh Jain Kanav Sharma

BITS Pilani · Nebula 2026, Software Category

September 2026

Abstract

Mixture-of-Experts models activate a small slice of their parameters per token, yet every expert has to sit in GPU memory. DeepSeek-V3 activates about 37 billion of its 671 billion parameters and still needs sixteen 80 GB GPUs just to hold its weights. We asked whether a larger, slower tier such as CXL-attached DRAM can hold the idle experts, and what that costs. To answer it we characterised the tier in DRAMSim3 and gem5, captured 258,629 tokens of real routing from OLMoE-1B-7B, built a trace-driven simulator, an event-driven SystemC model and a closed form for the critical batch size, and built MEMoE-RT, a runtime that runs real checkpoints with their experts outside GPU memory.

The measurements pushed us away from the usual popularity-based tiering design. At serving batch sizes a batch touches nearly every expert, fetch hit rate equals the resident fraction regardless of routing skew, and a placement profiled on one workload can serve another worse than random placement. What is left is a capacity decision and a transfer-scheduling problem. MEMoE-RT, built on that design, runs OLMoE-1B-7B with every expert off-GPU at a prefill batch of 16,384 tokens, keeping 91.2% of resident throughput in 7.27 GB instead of 17.73 GB. It also runs DeepSeek-V2-Lite (31.4 GB) on a 23 GB GPU at 7,241 tokens per second. In a serving loop the same offload adds 7% to prefill time and 525% to decode time, so our main recommendation is to use expert offload on prefill workers and keep decode resident. A planning tool carries the method to other checkpoints and GPUs: from a model config and two measurements of the card, it predicts what fits and where the crossover sits, and it places DeepSeek-V2-Lite’s crossover on the A10 inside the interval we measured.

1 The problem

A Mixture-of-Experts (MoE) layer replaces one feed-forward network with many experts, and a router picks a few of them per token. Compute per token stays roughly flat while parameter count grows. That works well on paper. In deployment, every expert still has to be addressable by the GPU, even though almost none of them are in use at any moment.

Table 1 shows how lopsided this is. In the four checkpoints we modelled, routed experts are 93.1% to 97.5% of all parameters. DeepSeek-V3 needs sixteen 80 GB devices to hold 1,249 GiB of weights, and 1,218 GiB of that is experts. Most of those GPUs are there for memory, not for compute.

Table 1. Weights and expert pool in GiB from our footprint model, which reproduces the published parameter counts. GPU counts assume 80 GB of HBM per device, plus 512 GB of CXL memory per device in the last column.

Model	Weights	Expert pool	Experts	GPUs, HBM only	GPUs, HBM+CXL
OLMoE-1B-7B [1]	12.9	12.0	93.1%	1	1
Mixtral-8x7B [2]	87.0	84.0	96.6%	2	1
Qwen3-235B-A22B [3]	437.8	423.0	96.6%	6	1
DeepSeek-V3 [5]	1249.2	1218.0	97.5%	16	3

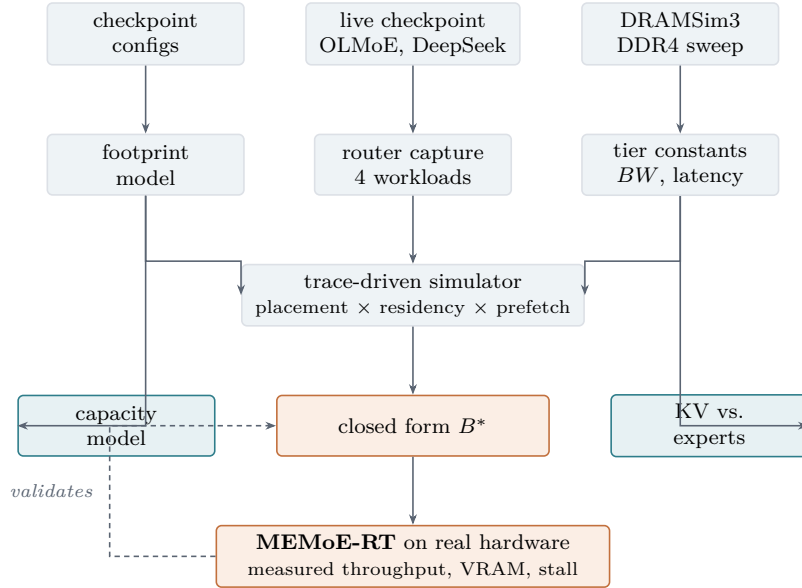
Compute Express Link (CXL) [6] is one way out. A CXL Type-3 expander attaches ordinary DRAM over a PCIe-class link and can add hundreds of gigabytes per node for much less than HBM costs. It is also far slower. The obvious design is to keep popular experts in HBM and fetch the rest on demand. We set out to build that, and our own measurements talked us out of it.

2 What we built

MEMoE is a measurement stack plus a runtime. Figure 1 shows how the parts connect, and Table 2 maps each deliverable in the problem statement to where it is covered.

Table 2. Deliverables and where they are addressed. All code is in the repository; paths are relative to its root.

Deliverable	Implementation	Section
HBM + CXL architecture model	<code>memoe/memory.py</code> , <code>configs/memory/</code>	§3
Transformer and MoE workload analysis	<code>router capture memoe/hooks.py</code> , <code>memoe/skew.py</code>	§4, §6
Bandwidth, latency, scalability	DRAMSim3 sweep, gem5 channel scaling	§3, §12
Tiering and expert placement	<code>memoe/policy.py</code> , <code>memoe/sim.py</code> , MEMoE-RT	§4–§7
Expansion and pooling study	<code>scripts/run_extras.py</code>	§12
Simulation framework, HBM-only vs HBM+CXL	simulator, SystemC, closed form, runtime	§8
Other models and GPUs	<code>scripts/plan_offload.py</code> , <code>docs/PORTING.md</code>	§14
Interactive dashboard	<code>results/dashboard.html</code>	§8
Recommendations		§16

**Figure 1.** How the components feed each other. Everything above the runtime is analysis or simulation; the runtime is the only part that touches real weights on a real device, and it is what checks the closed form against measured behaviour.

3 The two tiers

3.1 Architecture model

Everything in this report rests on a model of two memory tiers attached to one compute device (Table 3).

Table 3. The two tiers as modelled. HBM figures are HBM3e specifications at the effective rate we use throughout. CXL figures are our DRAMSim3 measurements at DDR4-3200 plus a controller and link allowance.

	HBM3e (fast tier)	CXL Type-3 DDR4 (capacity tier)
Attach	on-package, on the GPU	PCIe/CXL link, off-package
Capacity/device	80 GB modelled	512 GB per card, 2 TB pooled
Rated bandwidth	3,350 GB/s	25.6 GB/s per channel
Effective bandwidth	3,350 GB/s	16.88 GB/s measured, 67.5 GB/s for 4 channels
Loaded latency	~350 ns	179.4 ns device, ~300 ns total
Bandwidth vs HBM	1×	1/50 to 1/198
Expandable, poolable	no	yes (pooling needs a CXL 3.0 switch)

The bandwidth row is the design problem. A tier 50 to 198 times slower than HBM is hopeless for anything read on every decode step and fine for anything read once per batch. Section 6 turns that into a rule based on arithmetic intensity: expert weights can move, active KV cache cannot.

Latency matters less than it looks. The ~300 ns CXL figure is 179.4 ns of measured device latency plus about 120 ns for the controller and link hop, inside the 200–400 ns range reported for early CXL devices. When a whole expert moves in one transfer, latency is 0.16% of transfer time and the tier behaves as a pure bandwidth device. Section 12 shows where that stops being true.

3.2 What a CXL-class DRAM back end sustains

We didn’t want to quote CXL bandwidth from a datasheet. A Type-3 expander is DRAM behind a controller, so we simulated the DRAM in DRAMSim3 [7] and measured the rate it sustained while busy. Each trace was a 12 MB sequential read, the size of one OLMoE expert, issued as 196,608 accesses of 64 bytes. DRAMSim3’s headline bandwidth divides by the whole cycle budget including idle cycles, so we compute sustained bandwidth from the average interarrival time instead.

Table 4. DRAMSim3, DDR4 x8, 12 MB sequential expert read (four of the six grades swept).

Grade	Rated	Sustained	Share of rating	Loaded latency
DDR4-1866	14.9 GB/s	12.76 GB/s	85.5%	237.9 ns
DDR4-2400	19.2 GB/s	14.96 GB/s	77.9%	202.9 ns
DDR4-2933	23.5 GB/s	15.65 GB/s	66.7%	192.9 ns
DDR4-3200	25.6 GB/s	16.88 GB/s	65.9%	179.4 ns

We use 16.88 GB/s everywhere instead of the rated figure. The trend matters as well: the share of the rating actually reached drops from 85.5% to 65.9% as the grade gets faster, because timing constraints such as refresh don’t scale with the data rate. An expander should be sized by sustained bandwidth, and the gap to the rating widens with faster parts.

3.3 Channel scaling in gem5

The four-channel figure of 67.5 GB/s is the measured single channel times four. DRAMSim3 couldn’t measure it directly, because its trace driver issues at most one request per cycle and starves the extra channels. gem5 [8] has no such limit, so we ran a linear traffic generator through a CommMonitor into N interleaved DDR4-2400 MemCtrl channels (gem5 v22.1, 64-byte reads, 10 ms simulated, 128-byte bus).

Table 5. Sustained read bandwidth in gem5 against channel count.

Channels	Sustained	Ratio to one channel
1	13.86 GB/s	1.000
2	27.73 GB/s	2.001
4	55.13 GB/s	3.978
8	bus saturates; gem5 aborts on its 128-packet queue limit	

Four channels deliver 3.978 times one channel, linear to within 0.6%, which is what the four-channel figure needs. At eight channels the shared bus can’t drain responses fast enough and gem5 stops on its packet-queue sanity check. Four channels is the operating point this report uses. gem5’s single channel (13.86 GB/s) is also within 8% of DRAMSim3’s 14.96 GB/s for the same DDR4-2400 grade. Two simulators with different DRAM models land in the same place, well below the 19.2 GB/s rating.

4 How real routing behaves

Every tiering argument depends on how skewed routing is, so we measured it. We hooked the router of OLMoE-1B-7B-0924-Instruct and recorded the selected experts for every token across four workloads (prose, maths, code and chat), 258,629 tokens in total.

4.1 Skew depends on the input

Table 6. Routing statistics per workload, same model and weights. Normalised entropy of 1.0 is perfectly uniform.

Workload	Tokens	Zipf s	Entropy	Gini	Top quarter’s share
chat	18,239	0.494	0.972	0.261	40.7%
prose	37,355	0.598	0.963	0.300	43.5%
math	63,257	0.899	0.913	0.442	56.0%
code	139,778	1.132	0.877	0.522	63.6%

On identical weights, the Zipf exponent for code is 2.3 times that for chat. Chat routing is close to uniform: no expert gets more than 4.0% of routings, which is what the load-balancing loss used in training pushes toward. A system tuned for one fixed skew will be wrong for some of its traffic. Figure 2 shows what that skew is worth to a static placement: keeping the hottest quarter of experts resident serves 40.7% of routings for chat and 63.6% for code.

4.2 Coverage saturates at small batches

Tiering only pays if a batch leaves some experts untouched. In the captured traces, a batch of 64 tokens already touches 76% to 90% of experts per layer, and by 1,024 tokens it touches 97.3% for code and 99.4% to 100% for

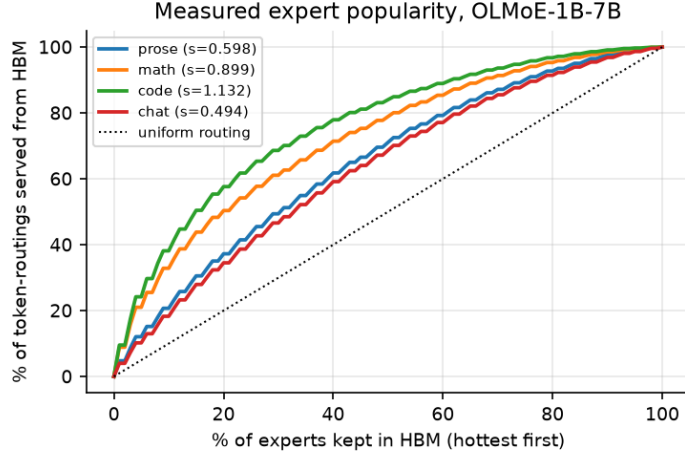


Figure 2. Share of token routings served from HBM when the hottest experts are kept resident, per workload, from the captured traces. The diagonal is uniform routing.

the others. Past that point every non-resident expert is fetched on every batch, so the fast tier is no longer a cache. It is a fixed partition. Real serving sits in that zone: prefill processes thousands of tokens at once, and continuous batching typically runs 64 to 256 sequences in decode.

4.3 Popularity buys nothing

If every expert gets touched, ranking experts by popularity shouldn’t help. We checked with the simulator, sweeping synthetic routing from uniform to more skewed than anything we measured, at 90% residency.

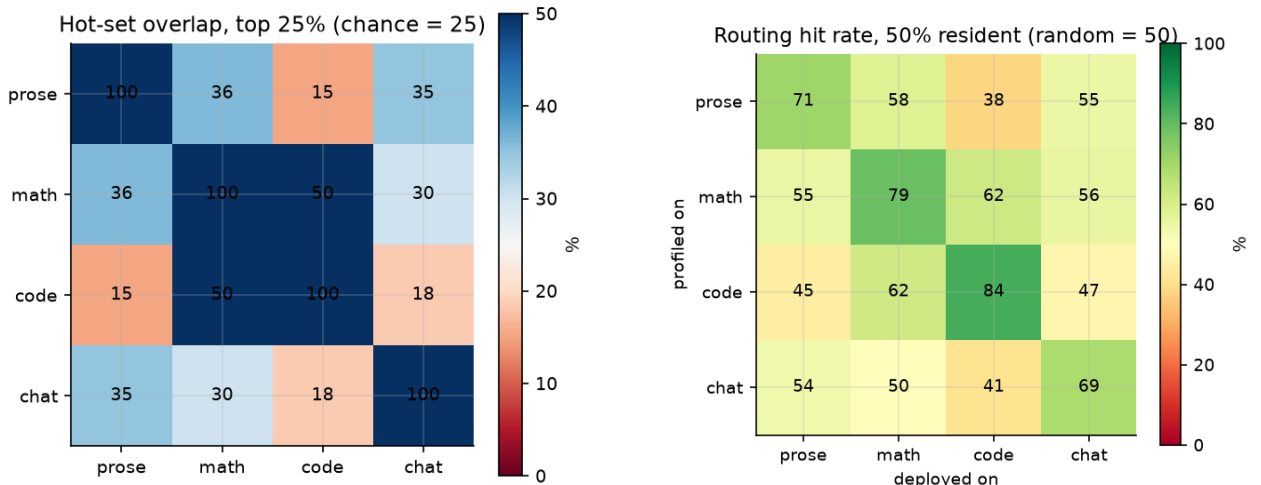
Table 7. Simulated fetch hit rate at 90% residency against routing skew.

Batch	$s = 0.0$	$s = 0.6$	$s = 1.0$	$s = 1.5$
64	0.899	0.900	0.907	0.927
512	0.899	0.899	0.899	0.899
8192	0.899	0.899	0.899	0.899

From batch 512 up, the fetch hit rate equals the resident fraction at every skew, and static placement, LRU and sampled LFU give the same number. That is good news for implementation: static placement is decided once at load time and costs nothing per token, so the cheapest policy is as good as any.

4.4 The wrong profile is worse than none

Placement can’t help at scale, but it can hurt at smaller batches, where the routing hit rate (the share of token routings served from HBM) still depends on which experts are resident. We took the top quarter of experts for each workload and measured overlap between workloads, with a split-half control inside each workload.



(a) Hot-set overlap. Chance is 25%.

(b) Routing hit rate at 50% residency. Random is 0.500.

Figure 3. Hot expert sets form two groups: prose with chat, and maths with code. Between the groups, overlap drops below chance.

Split-half overlap averages 84.9% (75.0% to 94.9%), so hot sets are stable within a workload. Across workloads, maths and code overlap 50.0%, but prose and code overlap only 15.2%, below the 25% you’d expect by chance. At 50% residency, a placement profiled on the workload it serves reaches a routing hit rate of 0.756 on average. A mismatched one averages 0.519, barely above random, and the worst pair (profiled on prose, serving code) reaches 0.383, which is worse than random. For mixed traffic there is no good static placement, so the resident set should be chosen by capacity alone.

5 The capacity model

With skew and policy out of the picture, two decisions remain: how much fast memory to keep, and how early to start each transfer. The first has a closed form. Let E be experts per layer, k experts activated per token, h the share of fetches the resident tier avoids, F per-GPU compute in FLOP/s, BW tier bandwidth, and ε the tolerated stall as a fraction of compute. Fetch traffic per layer scales with $(1 - h)E$ times expert size, and compute per layer scales with Bk times expert size. Expert size cancels, and transfer fits inside the stall budget once $B > B^*$:

$$B^* = \frac{(1 - h) E F}{BW \cdot k \cdot \varepsilon} \quad (1)$$

B^* doesn’t depend on expert size, since a bigger expert costs more to move and does proportionally more work once moved. It doesn’t depend on GPU count either, because compute and per-node bandwidth scale together under tensor parallelism. Adding GPUs doesn’t move the boundary.

Checking the closed form. Calibrating F on the fully resident OLMoE baseline gives 33.7 TFLOPS. With $h = 0.3$, $E = 64$, $k = 8$, $BW = 40.8$ GB/s and $\varepsilon = 1$, Equation 1 gives $B^* = 4,626$ tokens. From the runtime’s own measurements, the time to stream the bytes it actually moved at 70% offload equals resident compute time at 4,645 tokens, a 0.5% difference. The SystemC model, which shares no code with either, finds per-layer transfer and compute time equal at 4,626 (§13.1).

6 Experts, not the KV cache

The KV cache is the other thing that eats HBM, so we checked whether it belongs on the slow tier too. It doesn’t. A tier of bandwidth BW feeding a device at F FLOP/s is balanced at F/BW FLOPs per byte, and anything below that ratio is bandwidth-bound. Each fetched expert serves Bk/E tokens, so expert intensity *grows with the batch*. Each KV element is read once per decode step by $n_{\text{heads}}/n_{\text{kv heads}}$ query heads, so KV intensity is *fixed*, and batching can’t help because every sequence has its own cache.

Table 8. Arithmetic intensity in FLOPs per byte against a four-channel CXL tier, whose balance point is 5,861.

Model	KV intensity	Balance point	Batch at which experts balance
OLMoE-1B-7B	1.0	5,861	46,886
Mixtral-8x7B	4.0	5,861	23,443
Qwen3-235B-A22B	16.0	5,861	93,772
DeepSeek-V3	1.0	5,861	166,706

Qwen3 has the best KV intensity in the set thanks to 16-to-1 grouped-query attention, and it is still 366 times below the balance point. In concrete terms, one 32k-context Qwen3 sequence reads 5.875 GiB of KV per decode step. That takes 1.883 ms on HBM3e (531 steps per second) and 93.455 ms on a four-channel CXL tier (11 steps per second), a 49.6-fold slowdown. Active KV has to stay in HBM.

7 MEMoE-RT

Everything so far is analysis or simulation. To test it, we built a runtime that runs the real checkpoint with its experts somewhere else.

7.1 Design

Each finding above removes a component. Since coverage saturates, every expert in a layer will be needed, so there is nothing to predict and the only problem is starting the transfer early enough. MEMoE-RT therefore moves every non-resident expert of layer $L+d$ as one contiguous transfer, issued d layers ahead, on a dedicated CUDA copy stream that overlaps attention compute on the main stream. Since popularity doesn’t matter, the resident set is just the lowest expert indices, with no ranking. And since latency only matters for small transfers (§12), we move whole layers. We checked the link before building: 4 MB transfers reach 33.05 GB/s, 12 MB reach 36.67 GB/s, and larger ones plateau at 40.81 GB/s. A layer’s offloaded experts are hundreds of megabytes, well onto the plateau.

Offloaded experts live in pinned host memory, contiguous per layer, and a ring of $d+1$ GPU staging buffers receives them (Figures 4 and 5). Layer L uses slot $L \bmod (d+1)$, so the prefetch issued at layer L lands in the slot layer $L-1$ has just finished with. All waiting happens GPU-side through stream events, never host synchronisation, and stall time comes from CUDA events recorded around each wait. A second code path handles checkpoints that keep experts as a `ModuleList`, such as DeepSeek-V2-Lite: it leaves the model’s forward pass alone, points each offloaded expert’s weights at a fixed view into the staging ring, and drives transfers from hooks on each MoE block.

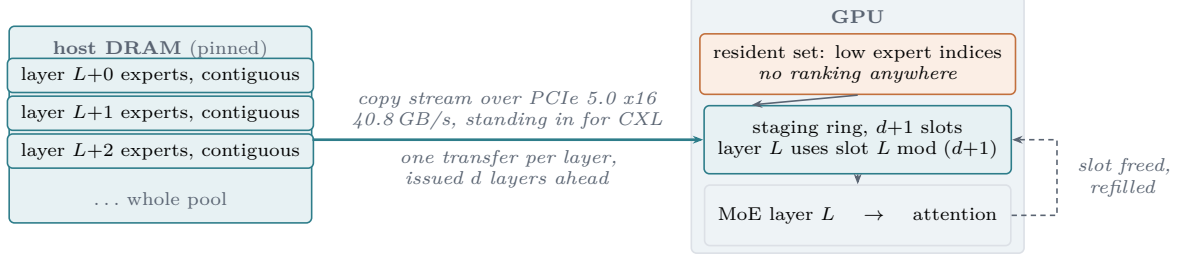


Figure 4. MEMoE-RT data path. Non-resident experts live contiguously in pinned host memory and move one whole layer at a time over a dedicated copy stream into a ring of staging buffers, d layers ahead of use. Waiting is done GPU-side with stream events, so the transfer overlaps attention compute rather than serialising with it.

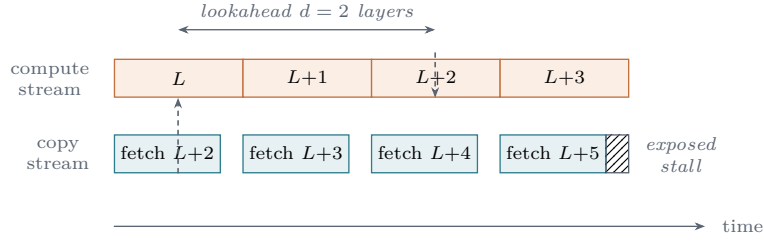


Figure 5. Two-stream execution with lookahead $d = 2$. The transfer for layer $L+2$ is issued while layer L computes, so a fetch has two layers of compute to hide behind. Roughly 60% of transfer time hides in practice; the hatched remainder is what the measured stall fraction reports.

7.2 PCIe as a stand-in for CXL

Host DRAM over PCIe is the standard stand-in for a CXL tier: a large, slow, byte-addressable tier behind a link. At PCIe 5.0 x16 its measured plateau of 40.8 GB/s sits between our single-channel CXL figure (16.88 GB/s) and the four-channel one (67.5 GB/s). MEMoE-RT moves whole layers in bulk, which depends on bandwidth rather than per-access latency or cache coherence, so PCIe reproduces the structure of the problem.

7.3 Correctness

Timing numbers mean nothing if offload changes the output, so every runtime result passes a check first. Running with 25% and 100% of experts resident on identical input gives a maximum absolute logit difference of exactly zero, so the transfer path changes nothing. Against the unmodified Hugging Face model, argmax agreement on random tokens is 92.8% on the Blackwell GPU, with 99.4% of tiered top-1 tokens inside the reference top-5 on the A10. The gap is bfloat16 rounding in our re-implemented MoE forward pass (CUDA `index_add_` accumulates in nondeterministic order). It is identical at every residency level, and half of random-token positions have a top-1/top-2 logit gap under 0.5.

8 Simulation framework: HBM-only against HBM+CXL

Every quantitative result compares two configurations of the same model on the same hardware. **HBM-only** has every weight, including the full expert pool, in GPU memory. **HBM+CXL** keeps only the non-expert weights and a resident fraction of experts in HBM, and streams the rest one layer ahead of use. Four implementations evaluate that comparison, and none of them share code (Table 9). DRAMSim3 and gem5 supply the tier constants to all four. The interactive dashboard ([results/dashboard.html](#), a single self-contained file) shows every result in this report and lets a reader enter their own model and hardware into Equation 1.

9 What the runtime measures

The OLMoE runs use bfloat16 on an RTX PRO 4500 Blackwell (32.6 GB, PCIe 5.0 x16), sequence length 512, three repetitions per configuration.

Table 9. The four evaluation paths.

Path	Implementation	What it gives
Analytical	<code>memoe/analysis.py</code>	B^* , capacity, pooling, KV intensity
Trace-driven	<code>memoe/sim.py</code> on 258,629 captured tokens	hit rate, coverage, policy comparison, placement penalty
Event-driven	SystemC TLM-2.0 [9], <code>systemc/memoe_tlm.cpp</code>	crossover and lookahead depth from a discrete-event model
Hardware	MEMoE-RT, two GPUs, two checkpoints	measured throughput, memory and stall on real weights

Table 10. Residency sweep at batch 4,096. Throughput is relative to the fully resident baseline of 20,917 tokens per second. Stall was not recorded for the first two offloaded rows.

Residency	Experts off-GPU	GPU memory	Tokens/s	Retained	Stalled
1.00	0%	14.84 GB	20,917	100.0%	0.0%
0.60	40.6%	10.59 GB	16,972	81.1%	–
0.40	59.4%	8.62 GB	15,062	72.0%	–
0.30	70.3%	7.48 GB	15,342	73.3%	3.2%
0.20	79.7%	6.50 GB	14,170	67.7%	6.8%
0.10	90.6%	5.35 GB	12,745	60.9%	15.2%
0.00	100%	4.37 GB	11,538	55.2%	21.2%

9.1 The cost of offloading

Run-to-run variation is about $\pm 3\%$, which is why the 0.30 row lands slightly above 0.40. At this batch size, offloading everything costs 45% of throughput. Batch size changes that picture completely.

9.2 The regime boundary

Table 11. Batch sweep with 70% of experts off-GPU in every row, against a resident baseline at the same batch size.

Batch	Baseline tok/s	Tiered tok/s	Retained	Stalled
512	5,340	2,046	38.3%	31.6%
1,024	9,194	3,771	41.0%	26.5%
4,096	20,917	14,614	69.9%	2.6%
16,384	26,873	25,367	94.4%	0.07%

The same configuration loses 62% of throughput at a decode-sized batch and 5.6% at a prefill-sized one. This is the measurement behind calling expert offload a prefill technique. Taking it further, we held *all* experts off-GPU at batch 16,384 and repeated the run three times from a fresh model load:

Headline. With all 64 experts of every layer outside GPU memory (93% of the model’s parameters), MEMoE-RT runs at 24,583 tokens per second against a resident baseline of 26,947, or **91.2%** (median of three trials, spread under 1.8%). GPU memory drops from 17.73 GB to 7.27 GB, a **2.44 $\times$ reduction**, and 0.56% of wall time is spent stalled on transfers.

9.3 Lookahead depth, and a correction

Our simulator had recommended prefetching 4 to 8 layers ahead. The hardware disagreed (Table 12). The simulator modelled per-expert on-demand fetches, which do need deep lookahead. The runtime moves a whole layer in one transfer, and at this operating point a layer’s transfer time and compute time are about equal, so one layer of lookahead is enough. Depth 8 uses 41% more staging memory for slightly lower throughput. We are reporting the correction rather than our original recommendation.

9.4 How much transfer hides under compute

The closed form assumes transfers hide completely under compute. Fitting measured wall time across the residency sweep to $t = t_{\text{compute}} + \kappa \cdot t_{\text{transfer}}$ gives $\kappa = 0.40 \pm 0.07$ (Figure 7): about 60% of transfer time hides and 40% is exposed, due to contention between the copy engine and compute. The ideal model overstates throughput by up to 40% where transfer and compute are balanced. Scaling the predicted exposure by κ brings it in line with the hardware across the sweep.

10 A second architecture that does not fit

To check that none of this is an OLMoE quirk, we ran MEMoE-RT on DeepSeek-V2-Lite [4]. It differs in almost every way that could matter: 26 MoE layers instead of 16, top-6 of 64 routed experts, two shared experts that

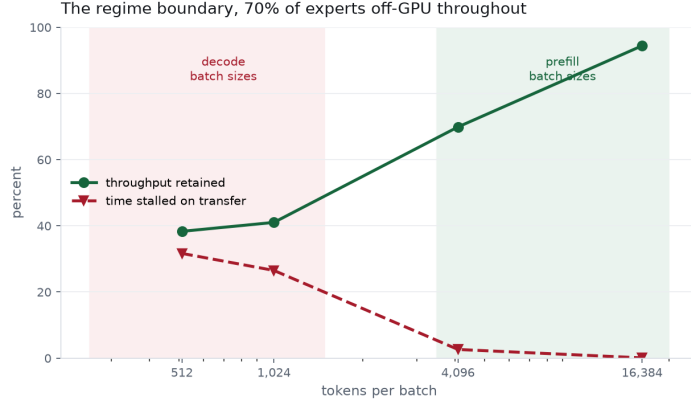


Figure 6. Retention and stall against batch size with 70% of experts off-GPU.

Table 12. Depth sweep at 59.4% offload, batch 4,096.

Lookahead	Tokens/s	GPU memory	Stalled
1	14,798	8.14 GB	2.4%
2	14,926	8.62 GB	2.3%
4	14,681	9.58 GB	2.3%
8	14,238	11.49 GB	2.3%

fire on every token, a dense first layer, multi-head latent attention, and its own modelling code. It also doesn’t fit. The weights are 31.4 GB, the expert pool is 28.8 GB (64 experts of 17.3 MB each per layer), and the NVIDIA A10 we used has 23 GB, so there is no resident baseline. All runs below keep every routed expert in host memory and stream it over PCIe 4.0 x16.

The boundary shows up from two directions. Stall drops from 48.4% at 4,096 tokens to 0.01% at 8,192. Achieved link bandwidth peaks at 24.03 GB/s at 6,144 tokens, where the link is fully busy and compute has just caught up. Beyond that the link is idle part of the time, falling to 12.72 GB/s at 16,384. So the same kind of crossover appears on a second architecture, over a link half as fast, at a different batch size. With compute calibrated on this model’s own run and the A10’s measured link of 25.2 GB/s, Equation 1 puts the crossover at 7,509 tokens, inside that interval (§14). OLMoE showed that a model that fits can run in less memory. DeepSeek-V2-Lite shows that a model that doesn’t fit can run at all.

11 What offload costs in a serving loop

Batch sweeps time single forward passes. Serving is different: requests arrive, prefill once, then generate a token at a time while other requests join and leave. We built a serving loop with a real KV cache and continuous batching and ran 32 requests of 2,048 prompt tokens each, generating 64 tokens, in three configurations: *resident* (every expert on the GPU), *offload* (every expert streamed, one engine for both phases) and *phase-aware* (prefill measured under offload, decode measured resident, as a disaggregated deployment would place them).

Streaming the whole expert pool adds 7% to prefill time and 525% to time per output token (Table 14). Swept across concurrency (Table 15, Figure 9), offloaded time per output token stays between 321 and 326 ms while concurrency grows eightfold, because every decode step streams the full expert pool however many sequences share it. Resident time per token grows linearly, fitting $9.8 + 1.33n$ ms, so the penalty falls from $21.8\times$

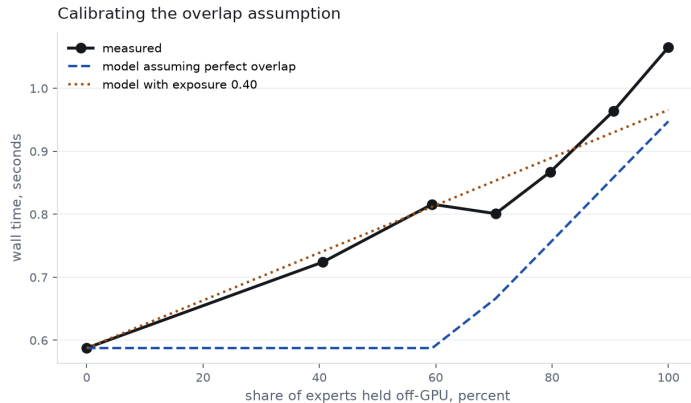


Figure 7. Measured wall time against the perfect-overlap model and the fitted one.

Table 13. DeepSeek-V2-Lite batch sweep on an A10 with the whole routed-expert pool in host memory. Every batch streams the same 86.4 GB over three repetitions, since coverage is total.

Batch	Tokens/s	GPU memory	Achieved link	Stalled
1,024	755	7.17 GB	21.22 GB/s	46.00%
4,096	2,772	10.16 GB	19.48 GB/s	48.38%
6,144	5,128	9.40 GB	24.03 GB/s	22.96%
8,192	6,572	10.54 GB	23.09 GB/s	0.01%
12,288	6,892	12.84 GB	16.15 GB/s	1.82%
16,384	7,241	15.13 GB	12.72 GB/s	0.01%

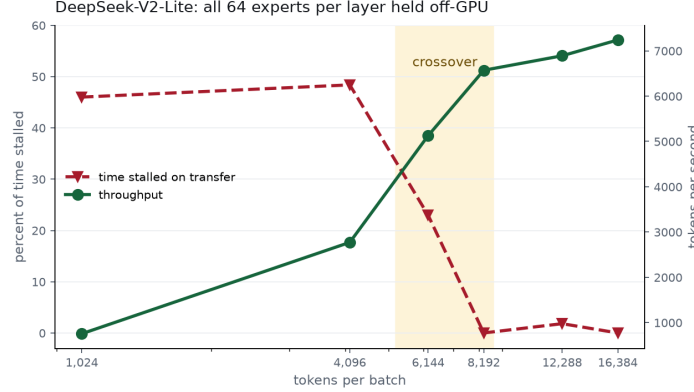


Figure 8. DeepSeek-V2-Lite: stall falls from 48% to near zero between 4,096 and 8,192 tokens.

to $6.25\times$ just by serving more sequences at once. Extending the fit, the two lines meet near 237 concurrent sequences, so offloaded decode moves closer to resident as concurrency rises.

Placed per phase, offload costs no measurable throughput: phase-aware serving produces 339.2 output tokens per second against 334.4 resident. The extra 1.4% is within run-to-run variation, so we read this as parity. Prefill-side GPU memory drops by 33%, from 31.59 GB to 21.12 GB. The two phases want opposite memory configurations, and a deployment that already separates them can take the saving on the prefill side without paying for it in decode.

12 Memory expansion, pooling and transfer granularity

Expansion is the case measured throughout this report: one expander per node holding that node’s expert pool turns a capacity problem into a bandwidth problem. Table 1 gives the effect at scale: DeepSeek-V3 goes from sixteen GPUs to three with 512 GB of CXL memory each. MEMoE-RT on DeepSeek-V2-Lite is the small-scale version of the same thing, running a model that otherwise wouldn’t run on that GPU at all.

Pooling puts one copy of the cold experts behind a switch shared by N nodes. The pool then needs $1/N$ of the capacity that dedicated expanders would, but all N nodes share one link, so per-node bandwidth falls as $1/N$. Because B^* is inversely proportional to bandwidth, the batch each node needs to stay above the crossover grows as N (Table 16).

At sixteen nodes the pool saves 6.3 TB, but each node would need a 1.5-million-token batch, which no deployment runs. Two to four nodes save real capacity while keeping the required batch closer to large prefill batches. Two things would change this tradeoff. A pool whose link bandwidth grows with membership (CXL 3.0 switching with multiple upstream ports) keeps B^* constant, but that hardware is not yet shipping, so our code marks every configuration that assumes it as modelled (`measured: false`). Interleaving across several expanders also raises per-node bandwidth, and our gem5 result supports near-linear gains up to four channels (§3.3).

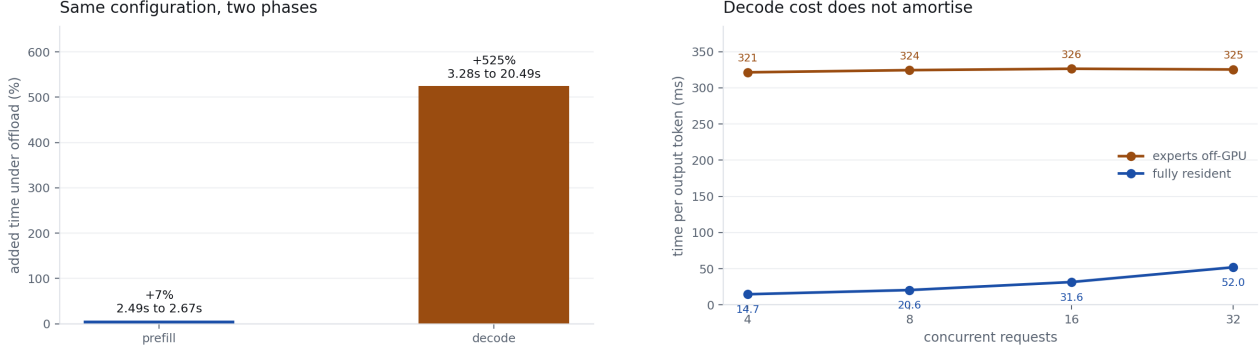
Transfer granularity. At 300 ns access latency and 67.5 GB/s, the share of transfer time spent on latency rather than moving bytes is 0.16% for a whole 12 MB expert, 3.72% for 512 KB chunks, 23.6% for 64 KB

Table 14. Serving at 32 concurrent sequences on the Blackwell GPU. All 32 requests are admitted before decoding begins, so time to first token reflects prefill only.

	Resident	Offloaded	Ratio
Prefill time	2.488 s	2.666 s	1.07
Time to first token	1,604 ms	1,728 ms	1.08
Time per output token	52.01 ms	325.31 ms	6.25
Output tokens per second	334.4	86.1	0.26
Peak GPU memory	31.59 GB	21.12 GB	0.67

Table 15. Time per output token (TPOT) against concurrent sequences.

Concurrency	Resident TPOT	Offloaded TPOT	Ratio	Offloaded VRAM
4	14.74 ms	321.38 ms	21.80	6.44 GB
8	20.55 ms	324.41 ms	15.79	9.48 GB
16	31.60 ms	326.32 ms	10.33	12.26 GB
32	52.01 ms	325.31 ms	6.25	21.12 GB



(a) The same offload in the two phases of a request.

(b) Offloaded TPOT is flat; resident TPOT grows.

Figure 9. Expert offload in a serving loop with continuous batching.

and 83.2% for 4 KB. Anything that breaks experts into pieces smaller than about 512 KB turns a bandwidth problem into a latency problem. There is a second cost too. Saturating a tier of bandwidth BW at latency L with requests of size S needs $BW \cdot L/S$ requests in flight, roughly 320 for a 67.5 GB/s tier at 300 ns with 64-byte accesses. Moving a whole layer as one DMA transfer avoids both. Our PCIe measurements show the same shape: 33.05 GB/s at 4 MB, rising to the 40.81 GB/s plateau.

13 Independent checks: SystemC and the Linux CXL stack

13.1 An event-driven model in SystemC

The closed form was calibrated against the runtime’s resident baseline, so those two aren’t fully independent. We therefore wrote the tiered path again as a small SystemC TLM-2.0 model that shares no code with either. A **MemoryTier** target returns latency plus bytes over bandwidth for each transaction. A **CopyEngine** issues one blocking transport per layer carrying all of that layer’s non-resident experts, and blocks when the staging ring is full. A **ComputeEngine** consumes layers in order and adds up the time it waits for a layer that hasn’t arrived. Nothing in the model knows about B^* .

The crossover appears where Equation 1 puts it: at 4,626 tokens a layer’s transfer and a layer’s compute both take 13.8 ms. Above the crossover, the remaining stall is a constant 13.81 ms at every batch size, exactly one layer transfer. That is the first layer, which nothing precedes and so can’t be prefetched. The small percentages in the table are that fixed cost divided by a growing wall time. Lookahead depths of 1, 2, 4 and 8 give identical throughput while staging memory grows from 1.13 GB to 5.07 GB, which matches what the hardware showed (§9.3). Sweeping tier bandwidth over 16.88, 40.8 and 67.5 GB/s gives crossovers at 11,180, 4,626 and 2,796 tokens, inversely proportional as the closed form says. The throughput ceiling of 20,746 tokens per second is within 1% of the measured resident baseline of 20,917.

13.2 The Linux CXL software path in QEMU

We also checked that the Linux CXL stack can manage this kind of tier, using QEMU 9.1 with an emulated Type-3 volatile memory device (2 GB) under an Ubuntu 24.04 guest. On the stock distribution kernel everything up to region commit worked: CXL control was negotiated through ACPI _OSC, the drivers bound, and the root, port and endpoint decoders enumerated with correct sizes. Committing a region then failed with **Failed to**

Table 16. Pooling Qwen3-235B-A22B experts behind a 67.5 GB/s pool, $h = 0.9$. Pooling figures are modelled: CXL 3.0 switches are not yet available to measure.

Nodes	Capacity saved	Per-node bandwidth	Required batch B^*
1	0 GB	67.50 GB/s	93,771
2	423 GB	33.75 GB/s	187,543
4	1,269 GB	16.88 GB/s	375,087
8	2,961 GB	8.44 GB/s	750,174
16	6,345 GB	4.22 GB/s	1,500,349

Table 17. SystemC batch sweep at 30% residency over a 40.8 GB/s tier, the runtime’s operating point.

Batch	Stall, % of wall	Tokens/s
256	94.5	1,154
1,024	78.2	4,570
4,096	16.1	17,561
4,626	5.9	19,698
16,384	1.7	20,566
32,768	0.9	20,746

synchronize CPU cache state.

The cause is a safety check, not missing support. Before committing a region the kernel must invalidate CPU caches over the range, and it gates that on `cpu_cache_has_invalidate_memregion()`, which returns false whenever `X86_FEATURE_HYPERVISOR` is set, i.e. inside any virtual machine. The kernel has a documented test bypass, `CONFIG_CXL_REGION_INVALIDATION_TEST`, which distribution kernels leave off. We built Linux 6.8 with it enabled, without root, and the region commits: `region0` is live at `0x490000000`, size 2 GB, one interleaved way targeting `decoder2.0` in `ram` mode, and the kernel logs `Bypassing cpu_cache_invalidate_memregion()` for testing.

This validates the software path a CXL tier depends on: discovery, decoder programming, region creation and commit. The performance numbers in this report come from DRAMSim3, gem5 and hardware.

14 Porting MEMoE-RT to other models and GPUs

Nothing in MEMoE-RT is specific to OLMoE or to our two GPUs. A new deployment takes a model config, two measurements of the card and one planning run, and the repository has a tool for each step (Table 18). `docs/PORTING.md` walks through them with DeepSeek-V2-Lite on the A10 as the example.

Table 18. Porting workflow.

Step	Tool	Output
Describe the model	YAML in <code>configs/models/</code> , filled from the checkpoint’s <code>config.json</code>	expert pool, weights that stay on the GPU
Describe the GPU	<code>scripts/measure_link.py</code> ; one run passed to <code>plan_offload.py -calibrate</code>	link bandwidth, effective FLOP/s, activation memory
Plan	<code>scripts/plan_offload.py</code>	largest residency that fits, B^* per residency, the command to run
Check and measure	<code>run_runtime.sh -check</code> , then a <code>-trials 3</code> sweep	PASS/FAIL, throughput, stall, peak memory

The runtime detects how a checkpoint stores its experts. Fused tensors (OLMoE, Qwen3-MoE and Mixtral in transformers 5.x) get a replacement MoE block; a `ModuleList` (Mixtral in transformers 4.x, DeepSeek-V2’s own code) gets weight views into the staging ring and transfer hooks. The launchers take the transformers and torch versions and the CUDA wheel index from the environment, so the same scripts run on the A10 with CUDA 12.1 wheels and on a Blackwell card with 12.8. For a model too large to load resident, the correctness check compares two residency levels instead of the unmodified model.

We verified the path on the A10. OLMoE-1B-7B and small randomly initialised Qwen3-MoE and Mixtral checkpoints, in both expert layouts, all pass the check, with offloaded output identical to the resident run. The `ModuleList` Mixtral is bit-identical to the unmodified model. For the planner, measuring the A10 gives a 25.2 GB/s link and, from a resident OLMoE run, 14.6 TFLOP/s of effective compute against 33.7 on the Blackwell card. Calibrated on committed runs, the planner reproduces $B^* = 4,625$ for OLMoE on the Blackwell card, and for DeepSeek-V2-Lite on the A10 it predicts 7,509 tokens (8,020 when calibrated at batch 12,288), inside the 6,144 to 8,192 interval where measured stall collapses from 23% to 0.01% (Table 13).

Table 19. Planner output for five model and GPU pairs (depth 1, 1.5 GB CUDA headroom). Card figures are measured; activation memory is calibrated where a run exists and 4 GB otherwise. B^* is the crossover with every expert streamed.

Model	GPU	Weights	Max residency	Host memory at max	B^* , all streamed
OLMoE-1B-7B	A10	13.8 GB	1.00 (resident)	–	4,631
OLMoE-1B-7B	RTX PRO 4500	13.8 GB	1.00 (resident)	–	6,608
DeepSeek-V2-Lite	A10	31.4 GB	0.45	15.8 GB	7,509
Mixtral-8x7B	A10	93.4 GB	0.10	81.2 GB	2,317
Qwen3-235B-A22B	RTX PRO 4500	470.1 GB	0.00	454.2 GB	13,216

Table 19 also shows how B^* follows the architecture. Mixtral’s 93 GB of weights fit a 23 GB card with 90% of experts streamed, and its crossover is low because it has few, large experts: $E/k = 4$, against 8 for OLMoE

and 16 for Qwen3. A card with less compute has a lower crossover too (OLMoE: 4,631 on the A10, 6,608 on the Blackwell card), since B^* is proportional to F/BW .

15 Related work

Expert offloading is a crowded area. The systems split mostly by the batch size they were designed for, which is the axis B^* describes. *Predict and cache* systems target single-sequence decoding: Mixtral-offloading keeps an LRU cache of experts and speculatively prefetches a layer or two ahead [10], MoE-Infinity adds request-level activation tracing [11], and ProMoE, HOBBIT and AdapMoE add learned prediction, mixed precision and expert skipping. Fiddler moves activations to the CPU and runs experts there, which it notes pays off when the batch is small [12]. *Throughput pipelines* work at large batches: ZeRO-Inference offloads layer weights with prefetch [13], FlexGen schedules across GPU, CPU and disk [14], and MoE-Lightning adds a CPU-GPU-I/O pipeline with a Hierarchical Roofline Model [15]. llama.cpp’s `-n-cpu-moe` option pins whole expert tensors to host memory by layer, with no ranking at all.

Our measurements explain why the first group’s machinery stops paying above B^* . Coverage saturates, so prediction becomes trivial. Hit rate equals residency, so LRU, LFU and static placement give the same result. And Fiddler’s condition that activations are small compared with expert weights is linear in batch size, so it flips at a batch set by the same quantities as B^* .

Several of MEMoE-RT’s pieces have precedent. Layer-granular prefetch on a separate stream is what ZeRO-Inference does, static unranked placement is what llama.cpp does, and using an analytical model to choose a policy is the idea behind MoE-Lightning’s roofline model. What we add is the boundary itself: a closed form for the batch size at which expert offload stops costing throughput, independent of expert size and GPU count, that someone can evaluate for hardware they don’t own, and that agrees with our runtime and SystemC model on OLMoE and with the measured crossover of DeepSeek-V2-Lite on the A10. We also give a measured reason why unranked static placement is the right choice at serving scale. Published speedups from these systems aren’t directly comparable with our retention figures, since each is measured against a different baseline.

16 Recommendations

1. **Put expert weights on the capacity tier, keep active KV cache in HBM.** What decides it is arithmetic intensity, not size: expert intensity grows with batch size, KV intensity doesn’t.
2. **Use expert offload for prefill.** At prefill batch sizes the whole expert pool can live off-GPU for under 10% of throughput. At decode batch sizes it costs most of it. In disaggregated serving, offload on prefill workers and keep decode workers resident.
3. **Size HBM with Equation 1, on measured card figures.** Measure the link and calibrate compute on the target GPU, then choose the resident fraction so the batches you run sit above B^* (`scripts/plan_offload.py` does both). Derate by $\kappa \approx 0.4$ for transfer that doesn’t hide under compute.
4. **Don’t build a popularity-based tiering engine.** At serving batch sizes fetch hit rate equals residency at every skew, and a placement profiled on the wrong workload does worse than random.
5. **Move one contiguous transfer per layer, one layer ahead.** Depth 1 matches depth 8 on throughput with 41% less staging memory, and chunks below about 512 KB become latency-bound.
6. **Specify expanders by sustained bandwidth.** Bulk reads reach only 66% to 86% of the DIMM rating, and faster grades fall further short.
7. **Keep pools small until switch bandwidth scales with membership.** Two to four nodes per pool today.

17 Conclusion

A slow memory tier can hold Mixture-of-Experts weights: during prefill, not during decode, and not because of caching. Coverage saturates, so the fast tier is a partition, and every popularity-based policy collapses to the same hit rate. What remains is a capacity decision, which has a closed form, and a scheduling problem, which moving whole layers one step ahead solves.

MEMoE-RT shows this on real hardware twice. On OLMoE-1B-7B, 93% of the parameters sit outside GPU memory at 91.2% of resident throughput for a $2.44\times$ memory reduction, and the closed form, the runtime measurements and an independent SystemC model agree on the crossover to within 0.5%. On DeepSeek-V2-Lite, a 31.4 GB model runs on a 23 GB GPU at 7,241 tokens per second, with its crossover where the closed form puts it. Because the boundary depends on bandwidth and compute rather than expert size or GPU count, the same analysis applies to a CXL expander once its measured bandwidth is substituted.

Code and data. Code, traces, results and the dashboard are at <https://github.com/AshBlake12/memoe>. `scripts/reproduce.sh` regenerates every analytical table and figure byte-for-byte on a CPU, with no GPU

or downloads. The runtime and serving results are reproduced on a GPU with `scripts/run_runtime.sh`, `run_serve.sh` and `run_deepseek.sh`, and `docs/PORTING.md` covers other models and GPUs.

References

- [1] N. Muennighoff et al. OLMoE: Open Mixture-of-Experts Language Models. arXiv:2409.02060, 2024.
- [2] A. Q. Jiang et al. Mixtral of Experts. arXiv:2401.04088, 2024.
- [3] Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025.
- [4] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434, 2024.
- [5] DeepSeek-AI. DeepSeek-V3 Technical Report. arXiv:2412.19437, 2024.
- [6] CXL Consortium. Compute Express Link Specification, Revision 3.1, 2023.
- [7] S. Li, Z. Yang, D. Reddy, A. Srivastava, B. Jacob. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. IEEE Computer Architecture Letters, 2020.
- [8] J. Lowe-Power et al. The gem5 Simulator: Version 20.0+. arXiv:2007.03152, 2020.
- [9] IEEE Std 1666-2011, IEEE Standard for Standard SystemC Language Reference Manual, 2012.
- [10] A. Eliseev, D. Mazur. Fast Inference of Mixture-of-Experts Language Models with Offloading. arXiv:2312.17238, 2023.
- [11] L. Xue et al. MoE-Infinity: Offloading-Efficient MoE Model Serving. arXiv:2401.14361, 2024.
- [12] K. Kamahori et al. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. ICLR 2025, arXiv:2402.07033.
- [13] R. Y. Aminabadi et al. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. SC 2022, arXiv:2207.00032.
- [14] Y. Sheng et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. ICML 2023.
- [15] S. Cao et al. MoE-Lightning: High-Throughput MoE Inference on Memory-Constrained GPUs. ASPLOS 2025, arXiv:2411.11217.